

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Smart Timetable Generator Portal

Project Name: Smart Timetable Generator Portal

Document: Software Requirements Specification

Project Type: Web-Based Application

Development Approach: Vibe Coding / Full Stack Development

Scheduling Approach: Rule-Based Scheduling

Hackathon: 2-Day Software Hackathon

Team Size: 4 Members

Date: 31 July 2026

1. Introduction

1.1 Purpose

The purpose of this document is to define the complete set of functional and non-functional requirements for the Smart Timetable Generator Portal. This Software Requirements Specification (SRS) serves as the primary reference for the development team, the hackathon evaluators, and future maintainers of the system. It captures the intended behaviour of the system in sufficient detail to guide design, implementation, and testing during the two-day hackathon.

1.2 Scope

The Smart Timetable Generator Portal is a web-based application that digitizes the manual process of academic timetable generation and management. The system provides centralized timetable management, role-based access control, rule-based (deterministic) timetable generation, automatic conflict detection, guest lecture handling, faculty absence handling with substitute faculty allocation, class adjustment tracking, report generation, notifications, and timetable search and filtering. The system does not use, and intentionally excludes, any artificial intelligence, machine learning, or predictive technologies.

1.3 Intended Audience

This document is intended for the following audience:

- The development team responsible for design and implementation.
- Hackathon evaluators assessing the Phase 1 submission.
- Faculty and administrative staff who will act as end users of the system.
- Future contributors who may extend the system beyond the hackathon scope.

1.4 Product Overview

The Smart Timetable Generator Portal replaces manual, spreadsheet-based timetable preparation with a centralized web portal. Administrators manage academic master data and generate timetables using a deterministic, rule-based scheduling engine. Faculty and students access timetables relevant to them through role-based, read-only views. The system also manages exceptional scenarios such as guest lectures and faculty absences, ensuring that the published timetable always reflects the current, conflict-free schedule.

2. Problem Understanding

Academic institutions traditionally prepare timetables manually using spreadsheets or paper-based methods. This approach is time-consuming and highly error-prone, particularly as the number of departments, faculty members, sections, and subjects increases. Manually cross-

checking faculty availability, classroom availability, and section schedules for every change is impractical at scale.

Handling day-to-day changes is especially difficult under the manual approach. Accommodating a guest lecture, adjusting the schedule for an absent faculty member, or reallocating a classroom typically requires manually re-checking the entire timetable for conflicts. Because there is no centralized system, faculty and students often lack timely access to the most current version of the timetable, and any modification must be manually communicated to all affected parties.

3. Major Problems Identified

#	Problem	Description
3.1	Manual Timetable Preparation	Timetables are prepared manually using spreadsheets, which is slow and error-prone.
3.2	Faculty Scheduling Conflicts	Faculty members are occasionally double-booked across sections due to the absence of automated validation.
3.3	Classroom Conflicts	The same classroom is sometimes assigned to more than one class in the same time slot.
3.4	Section Conflicts	A section may end up with two classes scheduled at the same time due to manual oversight.
3.5	Difficult Timetable Modification	Any change to an existing timetable requires extensive manual rework to avoid new conflicts.
3.6	Guest Lecture Conflicts	Accommodating guest lectures manually often disrupts existing scheduled classes.
3.7	Faculty Absence	There is no structured process to identify and assign a substitute faculty member when a faculty member is unavailable.
3.8	Pending Makeup Classes	Classes cancelled due to absence or rescheduling are not consistently tracked for makeup scheduling.
3.9	Lack of Centralized Access	Faculty and students do not have a single, reliable source to view the current timetable.
3.10	Difficulty in Searching Timetables	Locating a specific department, section, or faculty timetable manually is slow and inconvenient.
3.11	Manual Report Preparation	Reports such as faculty workload or room utilization are compiled manually, which is time-consuming.
3.12	Lack of Real-Time Updates	Changes to the timetable are not reflected to affected users in real time.

4. Target Users

The system defines three primary user roles, each with a distinct level of access:

Role	Description	Main Permissions
Administrator	Manages academic master data and oversees timetable generation and exceptions.	View, Edit, Manage
Faculty	Views the timetable relevant to their assigned subjects and sections.	View Only
Student	Views the timetable relevant to their department, section, and semester.	View Only

5. Proposed Solution

The Smart Timetable Generator Portal addresses the problems identified above through a centralized web application built around a deterministic, rule-based scheduling engine. Academic master data — departments, faculty, students, subjects, classrooms, sections, and time slots — is maintained centrally. The scheduling engine applies a fixed set of hard and soft constraints to generate a conflict-free timetable automatically.

The system also provides structured workflows for guest lectures and faculty absences, complete with substitute faculty allocation and makeup class tracking, along with role-based views, search and filtering, and automated report generation.

It is important to emphasize that the proposed solution uses only predefined scheduling rules and deterministic logic. The system does not use artificial intelligence, machine learning, or any predictive or recommendation-based technology at any stage.

6. Objectives

- Digitize the end-to-end process of academic timetable generation.
- Reduce the manual effort involved in preparing and maintaining timetables.
- Automatically detect and prevent scheduling conflicts.
- Provide role-based access for Administrators, Faculty, and Students.
- Provide centralized, always-current timetable viewing.
- Support timetable search and filtering by department, section, faculty, and more.
- Handle guest lectures without disrupting existing schedules.
- Allocate substitute faculty when a faculty member is absent.
- Track pending makeup classes arising from cancellations.
- Generate administrative and academic reports.

- Notify affected users when the timetable changes.

7. Identification of Core Features

This section identifies the core features of the Smart Timetable Generator Portal. Each feature is described along with the problem it addresses and its primary user.

#	Feature	Description	Problem Solved	Primary User
7.1	Role-Based Authentication	Secure login with distinct access levels for Administrator, Faculty, and Student.	Lack of centralized, secure access.	All Users
7.2	Academic Data Management	Centralized management of departments, faculty, students, subjects, classrooms, and sections.	Fragmented, spreadsheet-based master data.	Administrator
7.3	Rule-Based Timetable Generation	Deterministic generation of timetables using predefined hard and soft scheduling constraints.	Manual, error-prone timetable preparation.	Administrator
7.4	Conflict Detection	Automatic detection of faculty, classroom, and section clashes during generation and modification.	Faculty, classroom, and section conflicts.	Administrator
7.5	Timetable Search and Filtering	Search timetables by department, semester, section, faculty, academic year, or room.	Difficulty locating relevant timetables.	All Users
7.6	Guest Lecture Management	Structured workflow to schedule guest lectures while checking slot availability.	Guest lecture conflicts with existing classes.	Administrator
7.7	Substitute Faculty Allocation	Identification and assignment of an available substitute faculty member for absentee classes.	No structured process for faculty absence.	Administrator
7.8	Class Adjustment Management	Tracking of rescheduled, replaced, or cancelled classes, including pending makeup classes.	Untracked makeup classes.	Administrator
7.9	Reports	Generation of workload, utilization, and allocation reports with filtering support.	Manual report preparation.	Administrator, Faculty, Student
7.10	Notifications	Alerts sent to affected users whenever the timetable changes.	Lack of real-time updates.	All Users

8. Functional Requirements

The functional requirements of the system are listed below using clear, testable “shall” statements.

Req. ID	Requirement	Description	User
FR-01	Authentication	The system shall allow registered users to log in using valid credentials.	All Users
FR-02	Role-Based Authorization	The system shall restrict access to features based on the user's assigned role.	All Users
FR-03	Department Management	The system shall allow the Administrator to create, update, and remove department records.	Administrator
FR-04	Faculty Management	The system shall allow the Administrator to manage faculty records and availability.	Administrator
FR-05	Student Management	The system shall allow the Administrator to manage student records.	Administrator
FR-06	Subject Management	The system shall allow the Administrator to manage subject records and required weekly hours.	Administrator
FR-07	Classroom Management	The system shall allow the Administrator to manage classroom records, including capacity and type.	Administrator
FR-08	Course and Section Management	The system shall allow the Administrator to manage courses and sections.	Administrator
FR-09	Time Slot Management	The system shall allow the Administrator to configure available time slots for scheduling.	Administrator
FR-10	Timetable Generation	The system shall generate academic timetables using predefined rule-based scheduling constraints.	Administrator
FR-11	Conflict Detection	The system shall detect faculty, classroom, and section conflicts during timetable generation and modification.	Administrator
FR-12	Timetable Filtering	The system shall allow users to filter timetables by department, semester, section, faculty, academic year, and room.	All Users
FR-13	Guest Lecture Management	The system shall allow the Administrator to schedule guest lectures after checking slot availability.	Administrator
FR-14	Substitute Faculty Management	The system shall allow the Administrator to identify and assign substitute faculty for absent faculty members.	Administrator

Req. ID	Requirement	Description	User
FR-15	Class Adjustment	The system shall allow the Administrator to replace, reschedule, or cancel existing classes.	Administrator
FR-16	Makeup Class Tracking	The system shall record cancelled classes as pending makeup classes until rescheduled.	Administrator
FR-17	Report Generation	The system shall generate workload, utilization, allocation, and timetable reports.	Administrator, Faculty, Student
FR-18	Notifications	The system shall notify affected users when the timetable is modified.	All Users

9. Non-Functional Requirements

9.1 Security

- The system shall store user passwords using secure hashing (bcryptjs).
- The system shall use JSON Web Tokens (JWT) for authenticated session management.
- The system shall restrict data modification actions to authorized roles only.

9.2 Performance

- The system should generate a complete timetable within a reasonable processing time for typical institutional data volumes.
- The system should return filtered timetable views promptly under normal load.

9.3 Usability

- The system shall provide a simple, intuitive interface suitable for non-technical administrative staff.
- The system shall clearly present conflicts and validation errors to the Administrator.

9.4 Reliability

- The system shall ensure that a published timetable remains free of hard-constraint conflicts.
- The system should preserve data integrity during concurrent administrative updates.

9.5 Maintainability

- The system shall follow a modular backend structure (controllers, services, models) to ease future maintenance.

- The scheduling rules shall be centrally defined so that future rule changes require minimal code impact.

9.6 Scalability

- The system should support the addition of new departments, sections, and faculty without structural redesign.
- The database design should accommodate institutional growth over multiple academic years.

10. Scheduling Rules

The scheduling engine used by the Smart Timetable Generator Portal is rule-based and deterministic. Given the same input data and constraints, it will always produce the same conflict-free timetable. No artificial intelligence, machine learning, or probabilistic technique is used at any stage of scheduling.

10.1 Hard Constraints

Hard constraints must never be violated by a generated timetable:

- A faculty member cannot teach two classes simultaneously.
- A classroom cannot host two classes simultaneously.
- A section cannot attend two classes simultaneously.
- Faculty availability must be respected.
- Classroom availability must be respected.
- Required subject hours must be satisfied.
- Invalid time slots must not be assigned.
- Lab sessions requiring consecutive periods must receive consecutive slots.
- Room capacity and type must be suitable for the assigned class where applicable.

10.2 Soft Constraints

Soft constraints are optimized where possible, but may be relaxed if they conflict with a hard constraint:

- Avoid excessive consecutive classes for a faculty member.
- Distribute subject hours evenly across working days.
- Avoid unnecessary gaps in a section's schedule.
- Balance workload across faculty members.

- Prefer classrooms best suited to the class's requirements.
- Avoid repeatedly assigning the same subject in adjacent periods.

11. Guest Lecture Workflow

1. Administrator initiates the creation of a guest lecture.
2. Administrator selects the desired date and time.
3. System checks the availability of the requested slot.
4. If the slot is free, the system allocates it directly.
5. If the slot is not free, the Administrator chooses to replace, reschedule, or cancel the existing class.
6. If a class is cancelled, it is recorded as a pending makeup class.
7. The timetable is updated to reflect the change.
8. Affected users are notified of the update.

This workflow ensures that guest lectures are accommodated without silently overwriting an existing class: any conflict is surfaced to the Administrator for an explicit decision, and any resulting cancellation is tracked for follow-up rather than lost.

12. Faculty Absence Workflow

1. Faculty member applies for leave.
2. Administrator receives the leave request.
3. System helps identify available faculty for the affected slot.
4. System checks the candidate substitute for scheduling conflicts.
5. Administrator selects a suitable substitute.
6. The timetable is updated with the substitute assignment.
7. Students are notified of the change.

If no suitable substitute faculty member is available for the affected slot, the Administrator may mark the class for adjustment or schedule it as a pending makeup class.

13. User Permissions

Feature	Administrator	Faculty	Student
Login	✓	✓	✓
View Timetable	✓	✓	✓
Filter Timetable	✓	✓	✓
Generate Timetable	✓	X	X
Edit Timetable	✓	X	X
Manage Faculty	✓	X	X
Manage Students	✓	X	X
Manage Subjects	✓	X	X
Manage Rooms	✓	X	X
Guest Lecture	✓	X	X
Substitute Faculty	✓	X	X
Class Adjustment	✓	X	X
Reports (Generate/View)	✓	View Only	View Only
Notifications	✓	✓	✓

14. Timetable Generation Process

The timetable generation process follows a fixed, rule-based sequence and does not involve artificial intelligence at any step:

1. Collect input data.
2. Validate input data.
3. Load faculty availability.
4. Load subject requirements.
5. Load sections.
6. Load classrooms.
7. Load time slots.
8. Apply scheduling rules (hard and soft constraints).
9. Allocate available slots.
10. Detect conflicts.

11. Resolve or reject invalid assignments.
12. Validate the complete timetable.
13. Generate the final timetable.
14. Administrator reviews the generated timetable.
15. Publish the timetable.

15. Timetable Search and Filtering

All authenticated users can search and view timetables according to their assigned permissions. The system supports filtering by:

- Department
- Semester
- Section
- Faculty
- Academic Year
- Room

Example: a user can select Department → IT, Year → 3rd Year, Semester → 5, Section → A, and view the corresponding timetable directly.

16. Reports

The system shall provide the following reports:

Faculty Workload

Shows teaching hours and assigned subjects for faculty members.

Department Timetable

Displays timetable information for a selected department.

Room Utilization

Shows how classrooms and laboratories are being utilized.

Subject Allocation

Shows subject assignments across sections and faculty.

Complete Timetable

Provides a complete institutional timetable based on selected filters.

17. Scope of the System

17.1 In Scope

- Role-based authentication and authorization.
- Centralized academic master data management.
- Rule-based, deterministic timetable generation.
- Automatic conflict detection.
- Guest lecture scheduling.
- Faculty absence handling and substitute allocation.
- Class adjustment and pending makeup class tracking.
- Timetable search and filtering.
- Report generation.
- Notifications for timetable changes.

17.2 Out of Scope

- Artificial Intelligence
- Machine Learning
- AI-based timetable generation
- Chatbots
- Recommendation systems
- Predictive analytics
- External AI APIs

The current solution intentionally excludes AI-based functionality of any kind.

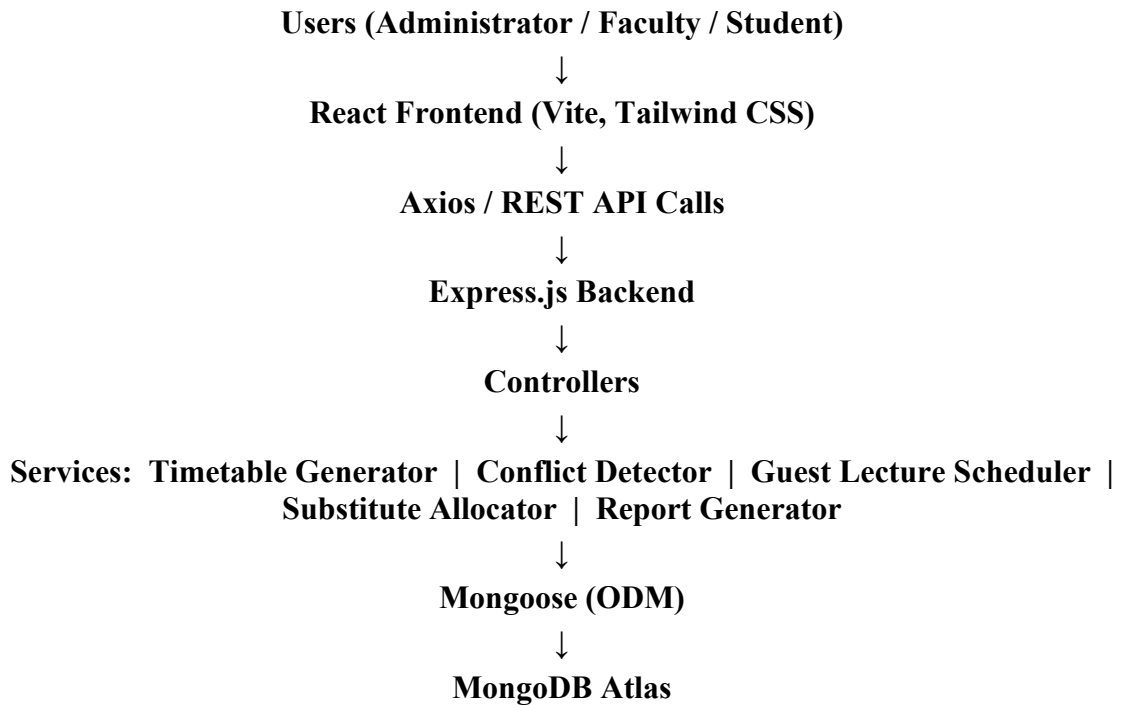
18. Technology Stack

Layer	Technology
Frontend	React, Vite, Tailwind CSS, Axios, React Router DOM
Backend	Node.js, Express.js
Database	MongoDB Atlas, Mongoose
Authentication	JWT, bcryptjs

Layer	Technology
Version Control	Git, GitHub
Deployment	Vercel (Frontend), Render (Backend)

19. High-Level System Architecture

The system follows a layered client-server architecture, illustrated below:



No AI-based service is present anywhere in this architecture. All services shown operate on deterministic, rule-based logic.

20. Major Database Entities

Entity	Purpose
Users	Stores login credentials and role information for all user types.
Departments	Stores academic department records.
Courses	Stores course/programme records offered by departments.
Sections	Stores section records within a course and semester.
Faculty	Stores faculty profiles, assigned subjects, and availability.
Students	Stores student profiles and section assignments.
Subjects	Stores subject details, including required weekly hours.

Entity	Purpose
Classrooms	Stores classroom details, including capacity and type.
TimeSlots	Stores the configured time slots used for scheduling.
Timetables	Stores the generated and published timetable data.
GuestLectures	Stores guest lecture requests and their allocated slots.
SubstituteFaculty	Stores substitute faculty assignments for absentee classes.
ClassAdjustments	Stores replaced, rescheduled, or cancelled classes and pending makeup classes.
Notifications	Stores notifications generated for timetable changes.

Detailed relationships between these entities will be represented in the Entity-Relationship (ER) Diagram during Phase 2 System Design.

22. Constraints

- Scheduling is rule-based only; no AI or ML techniques are used.
- The quality of the generated timetable depends on the accuracy of the input data.
- The hackathon implementation focuses on core scheduling workflows within the two-day timeframe.
- Advanced enterprise integrations are considered future scope.

24. Future Enhancements

The following items are identified as future enhancements and are not part of the current hackathon core implementation:

- Institutional ERP integration.
- Attendance system integration.
- Mobile application.
- Calendar integration.
- Advanced analytics.
- Automated email/SMS notifications.
- Historical timetable analysis.

25. Conclusion

The Smart Timetable Generator Portal replaces error-prone, manual timetable management with a centralized, role-based web application built on a deterministic, rule-based scheduling engine. By automating conflict detection and providing structured workflows for guest lectures, faculty absences, substitute allocation, and makeup class tracking, the system substantially reduces administrative effort while ensuring an accurate and consistently accessible timetable for all users.